

Продолжаем разбирать текстовые форматы на предмет получения текста. Итак, обещанный ранее PDF.

С portable document format'ом не всё так просто, как DOCX или ODT, что мы рассматривали в [прошлый раз](#), но всё же это всё ещё изначально текстовый, а не бинарный формат. Вы удивлены? Тогда давайте посмотрим на то, что там внутри. **Дальше действительно много текста.**

Как вы могли заметить, перед нами вполне себе «текстовый» документ, с вкраплениями бинарных данных. Конечно, как книгу pdf в блокноте не считаешь, но понимать, что написано и что в последствии будет отображено на экране, вполне возможно. Заранее отмечу, что целью этой статьи не является описание формата данных, поэтому буду рассказывать по существу: «Где искать текст?» Более подробную информацию по формату PDF вы найдёте по ссылкам в конце этого небольшого руководства.

## Типы данных PDF

PDF поддерживает несколько базовых типов данных (если быть точно восемь), часть из которых нам понадобится для работы — это строки (strings), массивы (arrays), словари (dictionaries), потоки (streams) и объекты (objects). Остановимся на каждом.

### Строки

Строки PDF унаследовал от PostScript, как следствие, под строкой в .pdf подразумевается последовательность 8-битных символов, окружённая круглыми скобками. String может перенесена на следующую строку с помощью обратного слэша, который не является частью строки и, помимо всего прочего, экранирует спецсимволы:

### Потоки

Потоки представляют последовательность восьмибитных данных между ключевыми словами stream и endstream. Любые бинарные данные, будь-то сжатый текст, изображение или внедрённый шрифт, будут представлены в виде потока. Поток всегда находится внутри объекта (чуть ниже) и характеризуется, как минимум, своей длиной (опция /Length N в словаре) и очень часто методом сжатия (например, /Filter /FlateDecode). PDF поддерживает достаточное количество форматов сжатия (в том числе и формат шифрования /CryptDecode), нас же будут интересовать лишь три: наиболее часто используемый Flate (gzip-сжатие) и более редкие ASCII Hex (представление данных в виде шестнадцатеричной строки с конечным символом >) и ASCII 85-based (сжатие, когда подряд идущие 4 символа исходного текста кодируются 5 символами от ! до u в ASCII таблице).

В stream'ах мы будем искать текст, который хотим получить из PDF-документа. Пример потока вы можете найти во второй половине изображения, что вначале данного топики: *да-да, те крякозябрики — это оно и есть.*

### Объекты

Объекты — это наибольшая структура, с которой на предстоит работать. Объект может содержать внутри себя любой другой тип данных от обычного числа до

потока, обрамляется ключевыми словами `obj` и `endobj`. Объект имеет свой ID внутри документа, по которому можно на него ссылаться. Нам в первую очередь интересны объекты с потоками внутри себя (не забываем об основной подзадаче), которые почти всегда содержат ещё и набор дополнительных опций в виде словаря. Вот обычный пример объекта внутри PDF-файла (с несжатым содержимым потока):